Diskbench: User-level Disk Feature Extraction Tool

Zoran Dimitrijević[1] Raju Rangaswami[1] Edward Chang[1]
David Watson[2] Anurag Acharya[2]

Univ. of California, Santa Barbara[1] Google[2]
{zoran,raju}@cs.ucsb.edu echang@ece.ucsb.edu {davidw,acha}@google.com

Despite the increases in disk capacity and decreases in mechanical delays in recent years, the performance gap between magnetic disks and CPU continues to increase. To improve disk performance, operating systems and file systems must have detailed information (e.g., zoning, bad-sector positions, and cache size) about the disks that they use. In this paper, we present Diskbench, a tool that we have developed to extract such information. Diskbench uses both interrogative and empirical methods for extracting various disk features. We present extraction results for several testbeds. From our empirical study, we conclude that intelligent data placement and access methods can be devised, by exploiting low-level disk knowledge, to improve disk performance.

1 Introduction

The performance gap between hard drives and CPU-memory subsystems is steadily increasing. In order to bridge this gap, the operating system must use intelligent disk management strategies [7, 8, 10, 16]. Most strategies assume that detailed disk parameters, such as zoning, bad-sector locations, and disk latency, can be obtained from the disk manufacturers. However, the information they provide can be imprecise and static. For instance, disk vendors usually give out only the maximum, minimum, and average data transfer rates and seek time. In addition, some dynamic information such as the locations of bad sectors cannot be known prior to actual use. As a consequence, the effectiveness of most traditional disk management strategies can be compromised.

For optimal disk performance, it is necessary to tune disk accesses to the requirements of the application by extracting the necessary disk features. For example, a multimedia streaming server must predict the hard disk performance so that it does not under-utilize disks while maintaining real-time streaming requirements. However, disk abstractions (e.g., SCSI and IDE interface) hide

low-level device characteristics from the operating system and virtualize the access to the device in the form of logical blocks. Such device abstractions make the task of tuning disk operation to match application requirements (and thus improve IO efficiency) difficult. In this paper we present Diskbench, a tool for the extraction of disk features. Diskbench consists of two applications, Scsibench and Idextract. Scsibench [6] runs in user space on Linux systems and accesses SCSI disks through the SCSI generic interface provided by Linux. It uses interrogative and empirical methods for feature extraction. Idextract uses Linux raw disk access and empirical methods to extract features from any disk-like device. Diskbench is open source and available for download [5].

Using Diskbench, we can obtain many disk features including 1) rotational time, 2) seek curve, 3) track and cylinder skew times, 4) caching and prefetching techniques, and 5) logical-to-physical block mappings. In addition, the access time (including seek time and rotational delay) between two disk accesses can be predicted with high accuracy. Scsibench also supports the execution of disk traces by providing an intuitive interface for executing primitive SCSI disk commands (e.g., read, write, seek, enable/disable cache, etc.) with an accurate timing mechanism.

Using knowledge about disk features and the trace support, system or application programmers can obtain the precise distribution of times spent by the disk performing various operations. Bottlenecks can thus be identified, and disk scheduling can be adjusted accordingly to utilize the disk more efficiently. Two such systems which currently benefit from using Diskbench are XTREAM and Virtual IO.

The XTREAM multimedia system [4] provides real-time video streaming capability to multiple clients simultaneously. For video playback, the disk management must guarantee that all IOs meet their real-time constraints. If the system does not have information about low-level disk features, it must assume the worst-case IO time or use statistical methods. These pessimistic and statistical estimates of disk drive performance lead to sub-optimal performance of the entire system. In contrast, XTREAM uses Diskbench to obtain the required disk features for making accurate admission control decisions.

Virtual IO [3] is an abstraction for disk IO, which provides highly preemptible disk access with little or no loss in disk throughput. Virtual IO relies on being able to predict disk behavior and performance accurately. The implementation of virtual IO was made feasible due to Scsibench, which extracts essential disk information so that accurate disk-performance prediction is possible.

In addition to these systems, Diskbench can also help in the design and implementation of complex disk scheduling algorithms [7, 8, 10, 16]. The processor-disk speed gap enables current systems to spend more CPU cycles to improve disk performance. In addition, the almost uniform disk geometry makes it possible to compress the disk map (refer to Sections 3.3) and store it entirely in main memory. Because of above trends, the system can afford employing more complicated and effective disk scheduling policies to improve overall system performance. Implementing these scheduling policies, however, requires knowing disk features precisely and in real-time. Diskbench provides a fast, convenient, automatic, and generic method for disk feature extraction.

The rest of the paper is organized as follows: In Section 2, we present several disk features that are of interest. In Section 3, we explain various feature extraction methods used in Diskbench. Section 4 presents the results obtained using our tool on several different disks. In Section 5, we compare our extraction methods with existing approaches. In Section 6 we make concluding remarks and suggest directions for future work.

2 Disk Features

Before we get into the details of disk features that are of interest when designing high performance systems, we provide a brief overview of the disk architecture. The main components of a typical disk drive are:

- One or more *disk platters* rotating in lockstep fashion on a shared *spindle*,
- A set of *read/write heads* residing on a shared arm moved by an *actuator*,
- *Disk logic*, including the disk controller, and
- *Cache/buffer memory* with embedded replacement and scheduling algorithms.

The data on the disk drive is logically organized into disk *blocks* (the minimum unit of disk access). Typically, a block corresponds to one disk *sector*. The set of sectors that are on the same magnetic surface and at the same distance from the central spindle form a *track*. The set of tracks at the same distance from the spindle form a *cylinder*. Meta-data such as error detection and correction data are stored in between regular sectors. Sectors can be used to store the data for a logical block, to reserve space for future bad sector re-mappings (spare sectors), or to store disk meta-data. They can also be marked as "bad" if they are located on the damaged magnetic surface.

The *storage density* (amount of data that can be stored per square inch) is constant for the magnetic surfaces (media) used in disks today. However, since the outer tracks are longer, they can store more data than the inner ones. Hence, modern disks do not have a constant number of sectors per track. Disks divide cylinders into multiple *disk zones*, each zone having a constant number of sectors per track.

The *rotational speed* of the disk is constant (with small random variations). Since the track size varies from zone to zone, each disk zone has a different *raw bandwidth* (data transfer rate from the disk magnetic media to the internal disk logic). The outer zones have a significantly larger raw disk bandwidth than the inner ones.

When the disk head switches from one track to the next, some time is spent in positioning the disk head to the center of the next track. If the two adjacent tracks are on the same cylinder, this time is referred to as the *track switch time*. If the tracks are on different cylinders, then it is referred to as *cylinder switch time*. In order to optimize the disk for sequential access, disk sectors are organized so that the starting sectors on two adjacent tracks are skewed. This skew compensates for the track or cylinder switch time. It is referred to as *track skew* and *cylinder skew* for track and cylinder switches respectively.

The *seek time* is the time that the disk arm needs in order to move from its current position to the destination cylinder. In the first stage of the seek operation, the arm accelerates at a constant rate. This is followed by a period of constant maximum velocity. In the next stage, the arm slows down with constant deceleration. The final stage of the seek is the settle time, which is needed to position the disk head exactly at the center of the destination track. Since the disk seek mainly depends on the characteristics of the disk arm and its actuator, the seek time curve does not depend on the starting and destination cylinders. It depends only on the seek distance (in cylinders).

The disk magnetic surfaces contain defects because the process of making perfect surfaces would be too expensive. Hence, disk low-level format marks bad sectors and skips them during logical block numbering. Additionally, some disk sectors are reserved as spare, to enable the disk to re-map bad sectors that occur during its lifetime. The algorithm for spare sector allocation differs from disk to disk. In order to accurately model the disk for intelligent data placement, scheduling, or even simple seek curve extraction, a system needs detailed mapping between the physical sectors and the logical blocks. In addition to mapping, a system must be able to query the disk about

re-mapped blocks. Re-mapping occurs when a disk detects a new bad sector.

The *disk cache* is divided into a number of *cache segments*. Each cache segment can either be allocated to a single sequential access stream or can be further split into blocks for independent allocation. The cache parameters of interest are the segment size, the number of cache segments, the segment replacement policy, prefetching algorithms, and the write buffer organization. Prefetching is used to improve the performance of sequential reads. The write buffer is used to delay the actual writing of data to the disk media and enable the disk to re-schedule writes to optimize its throughput.

3 Extraction Methods

In this section, we present methods for extracting certain disk features. We use a combination of interrogative and empirical methods. Interrogative methods use interrogative SCSI commands to inquire the disk about its features. Empirical methods measure completion times for various disk access patterns, and extract disk features based on timing measurements.

In some of our extraction methods we assume the ability to force access to the disk media for read or write requests. Most modern disks allow turning off the write buffer. In the case of SCSI disks, this can be done by turning off the disk buffers, or by setting the "force media access bit" in a SCSI command.

3.1 Rotational Time

Since modern disks have a constant rotational speed, if the interrogative SCSI command for obtaining rotational period (T_{rot}) is supported by the disk, it will return the correct value. In the absence of the interrogative command, we can also use the following empirical method to obtain T_{rot}: First, we ensure that read (or write) commands access the disk media. Next, we perform n successive disk accesses to the same block, and measure the access completion times. The absolute completion time for each disk access is:

$$T_i = T_{end_reading} + T_{transfer} + T_{OS_delay_i} \quad (1)$$

$T_{end_reading}$ is the absolute time immediately after the disk reads the block from the disk media. $T_{transfer}$ is the transfer time needed to transfer data over the IO bus. $T_{OS_delay_i}$ is the time between

the moment when the OS receives data over the IO bus, and the moment when the data is transfered to the user level Diskbench process. Since we need to wait for one disk rotation for each successive disk block access, we have:

$$T_{i+1} - T_i = T_{rot} + (T_{OS_delay_{i+1}} - T_{OS_delay_i}) \tag{2}$$

$$T_{n+1} - T_1 = n \times T_{rot} + (T_{OS_delay_{n+1}} - T_{OS_delay_1}) \tag{3}$$

The rotational period for modern disks is much greater than OS delays and other IO overheads (not including the seek and rotational times). Thus, we can measure the rotational period as:

$$T_{rot_measured} = T_{rot} + \frac{\Delta T_{OS_delay_{n+1,1}}}{n} = \frac{T_{n+1} - T_1}{n} \tag{4}$$

For large n, the error term ($\frac{\Delta T_{OS_delay_{n+1,1}}}{n}$) is negligible.

3.2 OS Delay Variations

In order to estimate variations in operating system delay for IO requests, we use the following method. First, we turn off all disk caching and disk buffering. Then, we read the same block on disk in successive disk rotations, as in the empirical method for extracting T_{rot}. We measure completion times for each read request.

Variations in the rotational period T_{rot} are negligible. Because of this, variations in $T_i - T_{i-1}$ from Equation 2 give us the distribution of $\Delta T_{OS_delay_{i,i-1}}$:

$$\Delta T_{OS_delay_{i,i-1}} = T_{OS_delay_i} - T_{OS_delay_{i-1}} \tag{5}$$

Thus, by measuring variations in $T_{i+1} - T_i$ from Equation 2, we can estimate variations in operating system delay.

3.3 Mapping from Logical to Physical Block Address

Modern SCSI disks implement SCSI commands for address translation (Send/Receive Diagnostic Command [14]) which can be used to extract disk mapping. However, in the case of older SCSI disks, or for disks where address translation commands are not supported (e.g. ATA disks), empirical methods are necessary.

3.3.1 Interrogative Mapping

Using the interrogative method, a single address translation typically requires less than one millisecond. But, since the number of logical blocks is large, it is inefficient to map each logical block. Fortunately, modern disks are optimized for sequential access of logical blocks. Additionally, most disks use the skipping method to skip bad sectors (instead of re-mapping them) during the low-level format. Due to this, logical blocks on a track are generally placed sequentially. Thus, we can extract highly accurate mapping information by translating just one address per track, except when we detect anomalies (tracks with bad blocks).

Since the number of re-mapped blocks is small compared to total number of blocks, we propose using two data structures to store mapping information obtained using the interrogative method. In the first data structure, we store the mapping in the form the disk had immediately after the low-level format. Since there are no re-mapped blocks, we simply store the starting logical block number and the track size (in blocks) for each track. The second data structure is used to store information about all re-mapped blocks. This way, we only need to update the second structure periodically, using interrogative SCSI commands.

Figure 1 presents a simplified algorithm for the interrogative extraction used in Diskbench. Using the SCSI command for physical-to-logical address translation, we extract the LBA for the first sector (sector zero) of each track. If sector zero is marked as bad, we continue performing address translation for subsequent sectors until we obtain a valid logical block number. When we come across a cylinder in which the number of sectors that lack a valid LBA (bad or spare blocks) is above a fixed threshold ($K_{threshold}$, we mark that cylinder as logically bad. We do not use these cylinders in our seek curve extraction method. Tracks which have a substantial number of blocks without a valid LBA are usually the ones containing mostly spare sectors.

3.3.2 Empirical Mapping

Empirical methods for the extraction of mapping information are needed for disks that do not support address translation. The empirical extraction method used in Diskbench follows. In the first step, we measure the time delay in reading a pair of blocks from the disk. We repeat this measurement for a number of block pairs, always keeping the position fixed for the first block in the pair. In successive experiments, we linearly increase the position of the second block in a pair.

Procedure: *Interrogative Mapping*

- **Variables:**

  cyl_num : Total number of cylinders
  $track_per_cyl$: Number of tracks per cylinder
  i : Cylinder number
  j : Track number
  $cyl_info[i]$: data structure for cylinder i info
  $cyl_info[i].logstart[j]$: Starting LBA for track j on cylinder i
  $cyl_info[i].logsize[j]$: Track j's size in blocks

- **Execution:**

```
for i = 0 to cyl_num do
   for j = 0 to track_per_cyl do
      for k = 0 to K_threshold do
         cyl_info[i].logstart[j] = phys_to_log(i,j,k)
         if valid(cyl_info[i].logstart[j]) then break
      if not_valid(cyl_info[i].logstart[j]) then
         mark track bad.
sort by cyl_info[i].logstart[j]
calculate cyl_info[i].logsize[j]
```

Figure 1: Interrogative method for disk mapping.

Using this method, our tool extracts accurate positions of track and cylinder boundaries. We now emphasize this method in detail. Time $T(i)$ defined in Equation 6 is the completion time measured at the moment when the user process receives data for logical block address i. (The variables on the right side of Equation 6 are defined in Section 3.1.)

$$T(i) = T_{end_reading} + T_{transfer} + T_{OS_delay} \tag{6}$$

The access time ($T_a(x,0) = T_{end_reading}(i) - T_{end_reading}(0)$) is the time needed to access block x after accessing block 0. It includes both seek time and rotational delay, but does not include transfer time and OS delay.

$$\Delta T(x,0') = T(x) - T(0')$$

$$\Delta T(x-1,0) = T_a(x-1,0) + (T_{OS_delay_{x-1}} - T_{OS_delay_0})$$

$$\Delta T(x,0') = T_a(x,0') + (T_{OS_delay_x} - T_{OS_delay_{0'}}) \tag{7}$$

$$\Delta = \Delta T(x,0') - \Delta T(x-1,0)$$

$$\Delta = T_a(x, 0') - T_a(x-1, 0) + (\Delta T_{OS_delay_{x,0'}} - \Delta T_{OS_delay_{x-1,0}}) \quad (8)$$

Equation 8 presents the first derivative of $\Delta T(x, 0)$ defined in Equation 7.

When the OS delay variations are small, and both blocks $x-1$ and x are on the same track, Δ is small, and proportional to $T_{rot}/track_{size}$. When the access can be performed without any additional rotational delay after seek, or with delay of a whole T_{rot}, Δ is proportional to $-T_{rot}+T_{rot}/track_{size}$. The sign of Δ depends on T_{OS_delay}. If x and $x-1$ are on different tracks, or cylinders, Δ is proportional to skew time T_{skew}, where T_{skew} is the track or cylinder skew time. Since the cylinder skew is usually larger than the track skew, we can use the positions of the skew times in Δ in order to find out accurate track and cylinder boundaries.

$$norm(\Delta) = (\Delta T(x, 0') - \Delta T(x-1, 0)) \bmod T_{rot} \quad (9)$$

Equation 9 defines the normalized first derivative from Equation 8. This way we eliminate the $-T_{rot}$ factor, and hence can automatically extract accurate disk mapping.

The $norm(\Delta)$ is useful for automatic extraction only if the OS delay is small compared to the skew times. Since the OS delay is a random variable, we perform several measurements whenever $|\Delta|$ is greater than a specific threshold ($0.02 * T_{rot}$) and stop the measurement when the difference between two consecutive Δs is less than a specific threshold (5%).

3.3.3 Disk Zones

Using the extracted disk mapping, Diskbench implements methods for the extraction of zoning information, including precise zone boundaries, the track and cylinder skew factors for each zone, the track size in logical blocks, and the sequential throughput of each zone. The algorithm used to extract zoning information scans the cylinders from the logical beginning to the logical end based on the disk mapping table. Due to the presence of bad and spare sectors, some tracks in a zone may have a smaller number of blocks than the others. Since we store only the track size (in logical blocks) for each track, we may detect a new zone incorrectly. In order to minimize the number of false positives, we use the following heuristics. First, we ignore cylinders with a large number of spare sectors. Second, during the cylinder scan, we detect a new zone only if the maximum track size in the current cylinder differs from the track size of the current zone by more than two blocks. Third, we detect a new zone only when the size of the new zone (in cylinders) is above a specific

threshold.

3.4 Seek Curves

Seek time is the time that the disk head requires to move from the current to the destination cylinder. We implement two methods for seek curve extraction. The first method uses the SCSI seek command to move (seek) to a destination cylinder. The second one measures the minimum time delay between reading a single block on the source cylinder and reading a single block on the destination cylinder to obtain the seek time. In order to find the minimum time, we can measure the time between reading a fixed block on the source cylinder, and reading all blocks on one track in the destination cylinder. The seek time is the minimum of the measured times. Since LBAs increase linearly on a track, we also implement an efficient binary-search method in order to find the minimum access time or seek time. $T_{seek}(x, y)$ returns seek time (in μs) between logical blocks x and y. This function is symmetrical, i.e., $T_{seek}(x, y) = T_{seek}(y, x)$. This seek time also includes the disk head settling time.

3.5 Predicting Rotational Delay

In order to optimize disk scheduling, the OS may use both seek and rotational delay characteristics of a disk [8, 16, 10, 11]. We can predict rotational distance between two LBAs using the following:

• mapping information extracted in Section 3.3, and

• skew factors for the beginning of each track, relative to a chosen rotational reference point.

In this paper we choose the disk block with LBA zero as the reference point. Let c_i be the track's cylinder number, t_j the track's position in a cylinder, and $track_{size}(c_i, t_j)$ the track's size in logical blocks. Let $LBA_{start}(c_i, t_j)$ be the track's starting logical block number, and T(LBA) the time after access to a specific LBA is completed. The skew factor of a track is defined as:

$$s_{c_i,t_j} = [T(LBA_{start}(c_i, t_j)) - T(LBA_0)] \bmod T_{rot} \tag{10}$$

If the number of spare and bad sectors is small, we can accurately predict the rotational distance between two LBAs (x and y) using the following equations:

$$X = T_{rot} \times \frac{[x - LBA_{start}(c_x, t_x)] \bmod track_{size}(x)}{track_{size}(x)} + s_{c_x,t_x} \tag{11}$$

$$Y = T_{rot} \times \frac{[y - LBA_{start}(c_y, t_y)] \bmod track_{size}(y)}{track_{size}(y)} + s_{c_y,t_y} \tag{12}$$

$$T_{rot_del}(y, x) = (Y - X) \bmod T_{rot} \tag{13}$$

Using the seek time $T_{seek}(y, x)$ defined in Section 3.4 and the rotational delay prediction from Equation 13, we can predict the access time to a disk block y after access to a block x, $T_a(y, x)$ as:

$$T_a(y, x) = T_{rot_del}(y, x) + T_{rot} \times \left\lceil \frac{T_{seek}(y, x) - T_{rot_del}(y, x)}{T_{rot}} \right\rceil \tag{14}$$

3.6 Read Cache Parameters

Most disk drives are equipped with a read cache. Read caches improve disk performance by allowing for data prefetching. We now present methods to extract the cache segment size, the number of segments, and the segment replacement policy.

3.6.1 Cache Segment Size

The method for extracting the cache segment size consists of three steps. First we read a few sequential disk blocks from a specific disk location. Next we wait for a long enough period of time (several disk rotations) to allow the disk to fill up the cache segment with prefetched data. Finally we read consecutive disk blocks occurring immediately after the first read and measure the completion time. If the block is in the cache, the completion time includes only a block transfer time from the cache to the OS through the IO bus and a random T_{OS_delay}. If the block is not in the cache, the completion time also includes seek time and rotational delay, as well as data transfer time. The seek time and rotational delay are the dominant factors in the IO completion time if present. We can thus detect the size of a cache segment by detecting the moment when the completion time includes mechanical seek and rotational delays.

3.6.2 Number of Cache Segments

In order to extract the number of disk cache segments, we need to be able to clear the cache. We clear the cache by performing a large number of random sequential reads for different logical blocks, which effectively clear the cache by polluting it. In this extraction method we linearly increase the number of sequential streams accessing the disk. In each iteration we do the following: For each stream we read a few blocks (from locations which are not used in cache pollution). Then we wait

a sufficient amount of time so that the disk can fill the cache segment with prefetched data. After this step, we assume that the disk cache has allocated one cache segment for each stream.

We perform read requests for all streams and measure completion times. If the completion times are smaller than a specific threshold, we assume that all blocks were in the cache, and that the number of cache segments is greater than or equal to the number of streams in this iteration. When we detect that one read request requires an amount of time that exceeds the threshold, we repeat the entire experiment to confirm that the excessive time is not caused by a large random OS delay. On confirmation, we deduce that the number of cache segments is equal to the number of streams in the previous iteration.

By changing the access pattern in the previous experiment, and noting which streams are not serviced from the cache, we can deduce the policy used for the cache segment replacement.

3.7 Write Buffer Parameters

Most disk drives are equipped with a write buffer to improve the disk write performance. If the disk has sufficient space in the write buffer, then the write command will be completed as soon as data is transferred to the disk's write buffer. The disk writes this data to the disk surface in an optimal manner at some later time.

Figure 2 presents an empirical method for extracting the write buffer size. Between iterations, we allow the disk to purge the contents of the write buffer to the disk media. Before issuing the write request, we also seek to a cylinder far away from the write request's destination. When the write request size is smaller than the write buffer, we expect that the write completion times will increase linearly, proportional to the throughput of the IO bus. When the write request size is greater than the write buffer, the completion time will incur seek and rotational delays. We can detect this using simple heuristics.

4 Experimental Results

Before we present our experimental results, we provide an overview of Diskbench. Diskbench consists of two separate tools: Scsibench and Idextract. The main components of Scsibench are the following:

• A user level SCSI command library (SCSI library) which is built on top of the Linux SCSI generic

Procedure: *Write Buffer Size*

- **Variables:**

  max_size : Maximal estimated write buffer size
  $start$: Starting LBA for write request
  far : LBA for a block with large $T_{seek}(start, far)$
  i : Write request size iteration
  T_1, T_2, T_{prev}: Time registers

- **Execution:**

  $T_{prev} = 0$
  for $i = 1$ to max_size do
    disk_seek(far)
    wait($20 \times T_{rot}$)
    T_1 = get_time()
    disk_write($start$, i)
    T_2 = get_time()
    if $T_2 - T_1 - T_{prev} > \frac{T_{rot}}{10}$ then
      return $i - 1$
    $T_{prev} = T_2 - T_1$

Figure 2: Empirical method for extracting write buffer size.

interface,

• Implementation of algorithms (described in Section 3) for extracting various disk features, and

• Support for execution of SCSI disk traces.

Scsibench runs as a user-level process on Linux systems, and uses the SCSI generic interface to talk to the SCSI disk. Using the command line arguments, a user can specify features to extract, or traces to execute.

Idextract runs as a user-level process using Linux raw disk support for accessing any disk. All extraction methods rely on using only read and write disk commands.

We now present experimental results for each disk feature described in Section 3 on three testbeds. The first testbed is a dual Intel Pentium II 800MHz machine with 1GB of main memory and a 9GB Seagate ST39102LW 10000 RPM SCSI disk (12 disk heads). The second testbed is an Intel Pentium III 800MHz machine with 128MB of main memory and an 18GB Seagate ST318437LW 7200 RPM SCSI disk (2 disk heads). The third testbed is Intel Pentium 4 1500MHz with 512MB of main memory and an 40GB WD400BB-75AUA1 7200 RPM IDE disk.

	Disk	T_{rot}(in μs)	RPM	Interrog.
1.	ST39102LW	5972.56	10045.94	10045
2.	ST318437LW	8305.83	7223.84	7200

Table 1: Rotational time for two testbed disks.

The first configuration is a typical SMP server system with a fast SCSI disk and a large number of tracks per cylinder. The second configuration is a typical workstation, with a large but slower hard disk (lesser rotation speed). The third configuration is slightly newer PC workstation with IDE disk. We present results for the IDE disk only for methods which differ from SCSI disk methods.

4.1 Rotational Time

Using Equation 4 to calculate the time required for a single rotation of the disk, we obtained rotational times for the two testbed configurations. These are presented in Table 1.

4.2 Variations in OS Delay

```
B 0       ; Turn off all disk buffering
R 0 1     ; Read one sector starting from LBN 0
T 2       ; Store current time to register T2

          ; repeat the following:
R 0 1     ; Read one sector starting from LBN 0
T 3       ; Store current time to register T3
- 0 3 2   ; Print T3 − T2
- 2 3 0   ; T2 = T3 − 0
...
```

Figure 3: Sample trace file to find T_{OS_delay} variations.

Based on Equation 2, we note that the variation in operating system delay for disk accesses is proportional to the variation in completion times for the same request. Using the trace execution described in Figure 3, we measured variations in request completion times (and thus, the distribution of operating system delay variations) for the two testbed configurations. These are presented in Figures 4 and 5.

Figure 4 shows the results for our first testbed configuration. We can see that the variations in

OS delay are of the order of $10\mu s$. Figure 5 shows results for our second testbed configuration. Here the OS delay variations are of the order of $40\mu s$, with greater variations in OS delay as compared to the first testbed.

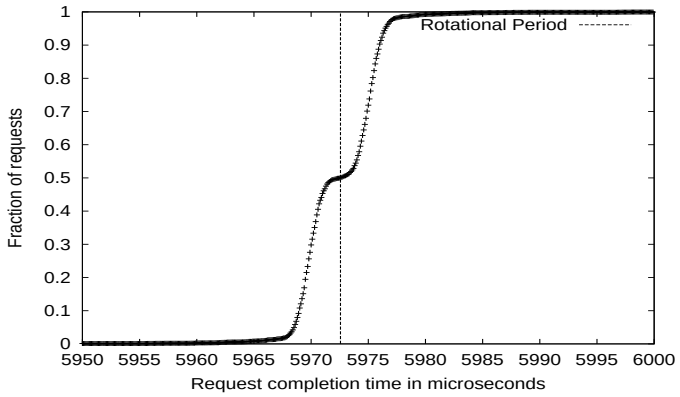


Figure 4: Distribution of request completion times for Seagate ST39102LW.

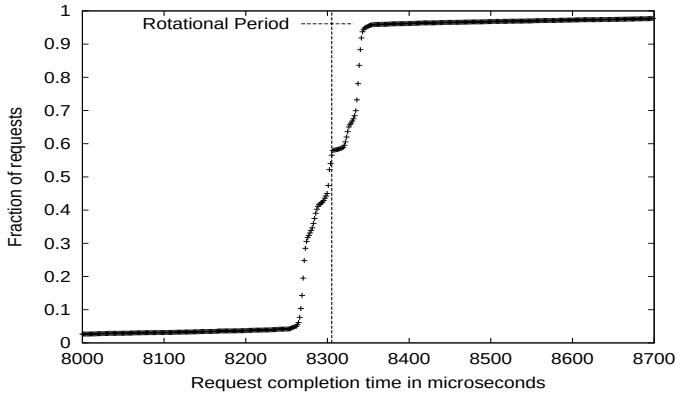


Figure 5: Distribution of request completion times for Seagate ST318437.

4.3 Mapping from Logical to Physical Block Address

4.3.1 Interrogative Mapping

Sample results from the mapping extraction for our testbed 2 configuration are presented in Figure 6. In each line we print the mapping information for one cylinder. After C we print the cylinder number, the starting LBA, and the cylinder size (in logical blocks). Then we print information for individual tracks after T, namely the track number, its starting LBA, and size (in logical blocks). We print this information for all tracks.

C 0 0 1500 T 0 0 750 1 750 750
C 1 1500 1500 T 0 2250 750 1 1500 750
C 2 3000 1500 T 0 3000 750 1 3750 750
C 3 4500 1500 T 0 5250 750 1 4500 750
C 4 6000 1500 T 0 6000 750 1 6750 750
C 5 7500 1500 T 0 8250 750 1 7500 750
C 6 9000 1500 T 0 9000 750 1 9750 750
...
C 718 1077000 1499 T 0 1077000 750 1 1077750 749
C 719 1078499 1499 T 0 1079248 749 1 1078499 750
C 720 1079998 1499 T 0 1079998 750 1 1080748 749
C 721 1081497 1499 T 0 1082246 749 1 1081497 750
...

Figure 6: Sample LBA-to-PBA mapping for Seagate ST318437LW.

Diskbench stores the starting LBA and the size of each disk track. If the number of bad sectors in a track is greater than the number of spare sectors allocated per each track, then the track size is smaller than the size of a regular track in that zone (cylinders $718 - 721$ in Fugure 6).

4.3.2 Empirical Mapping

We present results for the empirical extraction of mapping information for testbed 1 in Figures 7-10. This disk has 12 tracks per cylinder. In Figure 8 we present the access time $\Delta T(x, 0)$ (defined in Equation 7) between disk blocks 0 and x. The rotational period of the disk (T_{rot}) is approximately $6ms$. We detail our results in Figure 7, which is an enlargement of small section of Figure 8.

We can see that for small values of x, the access time $\Delta T(x, 0)$ is larger than T_{rot}. When T_{OS_delay} (defined in Section 3.1) is larger than the rotational distance between blocks 0 and x, the second read request (to the logical block x) has to be serviced during the next disk rotation. When $\Delta T(x, 0)$ is greater than T_{OS_delay}, an additional disk rotation is not needed. $\Delta T(x, 0)$ increases linearly for all blocks on the same track. When blocks $x - 1$ and x are located on different tracks, $\Delta T(x, 0)$ increases by the track (or cylinder) skew time (after which $\Delta T(x, 0)$ continues to increase linearly). In our example this happens at logical block number 254.

When the access time to the block $x-1$ requires a rotational delay of nearly T_{rot}, and the access

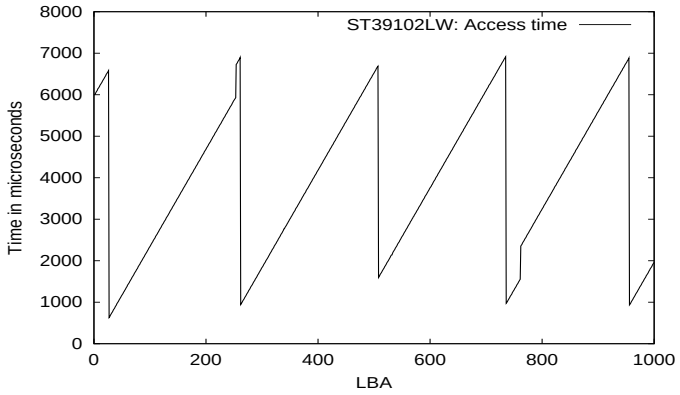


Figure 7: A sample of $\Delta T(x, 0)$ needed to read logical block x (on the X-axis) after reading LBA_0 for the ST39102LW.

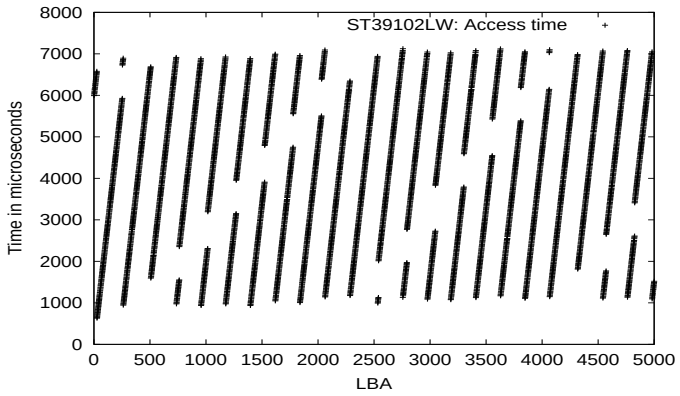


Figure 8: The time $\Delta T(x, 0)$ needed to read logical block x (on the X-axis) after reading LBA_0 for the ST39102LW.

to x does not require any rotational delay after seek, $\Delta T(x,0)$ decreases by T_{rot}. This happens at block number 262 for the first time. At the next track boundary (508), a skew time increase and a T_{rot} decrease overlap. Figure 8 shows the $\Delta T(x,0)$ curve for the distances up to 5000 logical blocks.

Figure 9 shows the first derivative of the $\Delta T(x,0)$ curve (Δ) defined in Equation 8. We can see that, since T_{OS_delay} is a random variable, $\Delta T(x,0)$ can also incur a sudden increase of T_{rot} in successive measurements. In this experiment, it occurs at $x = 2758$. This happens when the difference between T_{OS_delay} in successive measurements is substantial, so that $\Delta T(x,0)$ incurs one disk rotation more than $\Delta T(x-1,0)$. A positive T_{rot} increase in $\Delta T(x,0)$ is always followed by a negative T_{rot} decrease in the next few accesses.

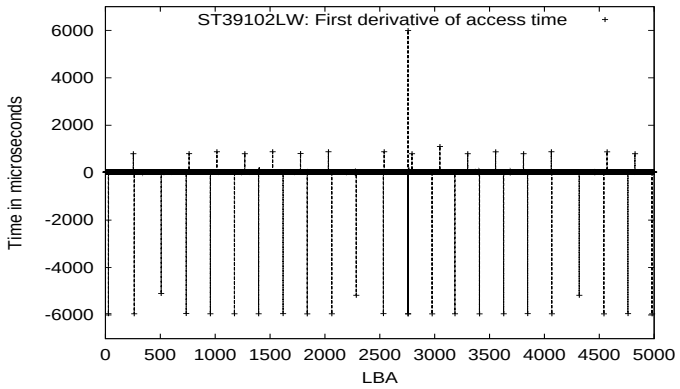


Figure 9: First derivative (Δ) of access time($\Delta T(x,0)$) for ST39102LW.

In order to perform the empirical mapping automatically, we use several heuristics to find the normalized value for Δ ($norm(\Delta)$) defined in Equation 9. Figure 10 presents $norm(\Delta)$ for our testbed 1, where we capture only the track and cylinder skew times. The positions of the track and cylinder skew times on the x-axis are exact positions of the track and cylinder boundaries (occurring every 254 blocks in Figure 10). The track and cylinder skew times for this disk are approximately $880\mu s$ and $1100\mu s$ respectively. The skew times to switch from an odd to an even track, and from an even to an odd track, are also slightly different ($880\mu s$ and $800\mu s$ respectively).

Figure 11 presents the empirical mapping results for testbed 2. The disk used in this configuration has two tracks per cylinder. Results from Section 4.2 show that variations in operation system delay, and hence the noise in the measured $norm(\Delta)$, is much higher than for the first testbed

configuration. However, since T_{OS_delay} is a random variable, we can repeat the experiment to limit the noise level and extract $norm(\Delta)$ accurately.

Figure 12 presents the empirical mapping results for testbed 3. We can see that Idextract results are similar to Scsibench ones. For this particular disk we are not able to find out the number of tracks per cylinder since the track skew is nearly identical to the cylinder skew time.

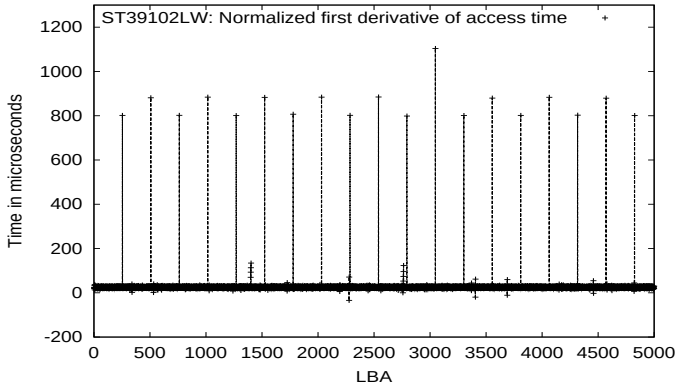


Figure 10: Normalized first derivative ($norm(\Delta)$) for ST39102LW.

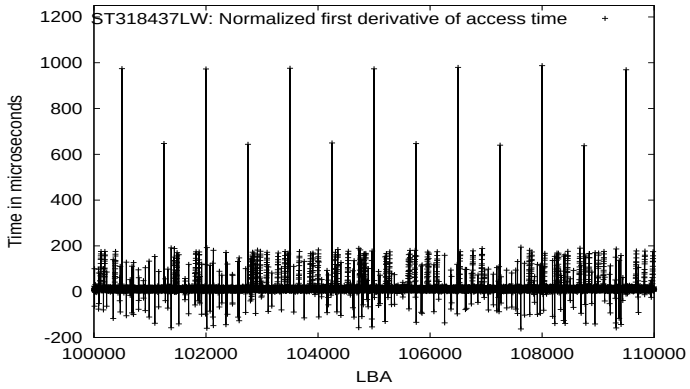


Figure 11: Normalized first derivative $norm(\Delta)$ for ST318437LW.

4.3.3 Disk Zones

Tables 2 and 3 present the zoning information extracted for the Seagate ST39102LW and Seagate ST318437LW disks respectively. t_{size} denotes the track size in logical blocks. $\mathcal{TR}$ is the transfer rate measured for long sequential reads that span multiple cylinders. $\mathcal{TR}_{max}$ is the calculated theoretical maximum transfer rates for read requests which incur no seek, rotation or switching

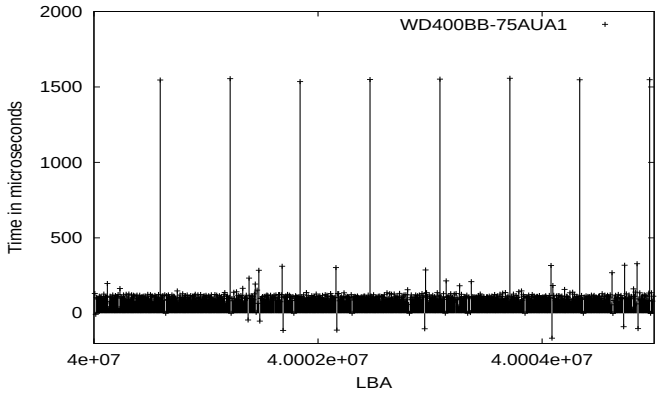


Figure 12: Normalized first derivative $norm(\Delta)$ for IDE WD400BB-75AUA1.

Zone	*Cylinders*	t_{size}	$\mathcal{TR}$	$\mathcal{TR}_{max}$	$\gamma(1)$	$\mathcal{H}$
1	0-847	254	18.85	21.77	1108	884
2	848-1644	245	18.02	21.01	1108	885
3	1645- 2393	238	17.51	20.40	1098	876
4	2394- 3097	227	16.70	19.45	1114	891
5	3098- 3758	217	15.99	18.60	1115	890
6	3759- 4380	209	15.43	27.91	1105	885
7	4381- 4965	201	14.84	17.23	1100	876
8	4966- 5515	189	13.98	16.20	1123	901
9	5516- 6031	181	13.39	15.52	1125	903
10	6032- 6517	174	12.89	14.92	1107	885
11	6518- 6961	167	12.38	14.31	1118	899

Table 2: Disk zone features for ST39102LW.

Zone	*Cylinders*	t_{size}	$\mathcal{TR}$	$\mathcal{TR}_{max}$	$\gamma(1)$	$\mathcal{H}$
1	0- 4553	750	31.07	46.24	977	652
2	4554- 6582	687	28.94	42.35	985	654
3	6583- 8247	678	28.48	41.80	987	648
4	8248-11554	666	27.92	41.06	1134	651
5	11555-14597	625	27.13	38.53	981	646
6	14598-17370	600	26.00	36.62	983	652
7	17371-19908	583	25.44	35.94	987	657
8	19909-22226	550	24.49	33.91	982	648
9	22227-26338	500	22.74	30.82	982	650
10	26339-28170	458	21.04	28.23	1159	654
11	28171-29850	437	20.24	26.94	990	663

Table 3: Disk zone features for ST318437LW.

overheads. $\mathcal{H}$ and $\gamma(1)$ are the track and cylinder switch times in microseconds.

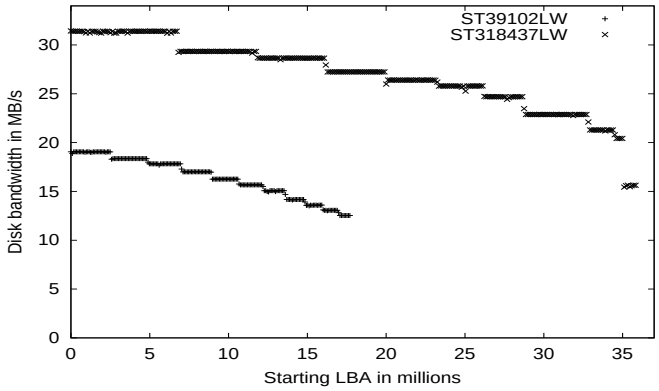


Figure 13: Disk bandwidth depending on data location for two SCSI disks.

Figure 13 depicts the disk bandwidth for large sequential accesses depending on the starting LBA for testbed 1 and 2. For modern disks, the difference between the maximum and minimum sequential disk bandwidth is usually a factor of two. Figure 14 presents the disk zone bandwidths for testbed 3.

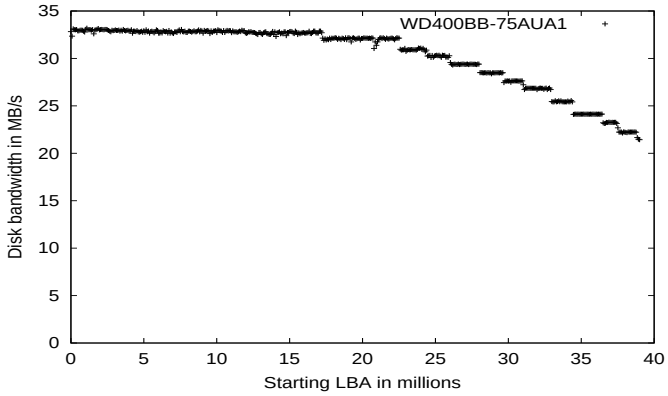


Figure 14: Disk bandwidth depending on data location for an IDE disk.

4.4 Seek Curves

In Figure 15 we present the seek curve for our first testbed (ST39102LW). We can see that the difference between the rotational period and the maximum seek time is less than a factor of two. Since the variations in the seek curve are negligible, we can also deduce that the seek time depends mainly on the seek distance in cylinders, and not on starting or destination cylinder positions. Figure 16 presents the seek curve for the second testbed.

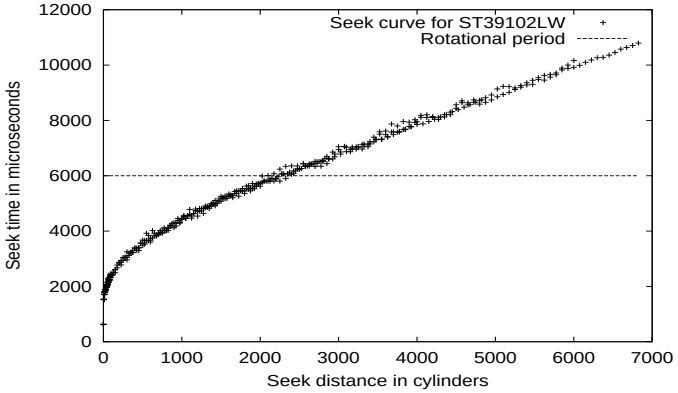


Figure 15: Complete seek curve for ST39102LW.

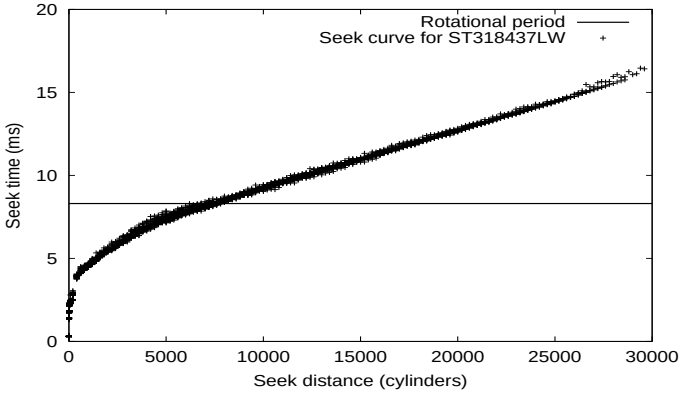


Figure 16: Complete seek curve for ST318437LW.

4.5 Rotational Delay Prediction

Cyl	$Track$	$Skew(\mu s)$	LBA_{rot0}
0	0	0	0
0	1	755.15	475
0	2	1595.02	694
0	3	2352.35	915
0	4	3191.09	1134
0	5	3949.03	1356
0	6	4788.61	1574
0	7	5546.36	1796
0	8	414.31	2268
0	9	1170.86	2490
0	10	2009.76	2708
0	11	2767.37	2930
1	0	3829.42	3139
...	...	...	...

Table 4: Rotational delay modeling for ST39102LW. Disk blocks with LBA_{rot0} are on the same disk radius. The track size for this zone is 254.

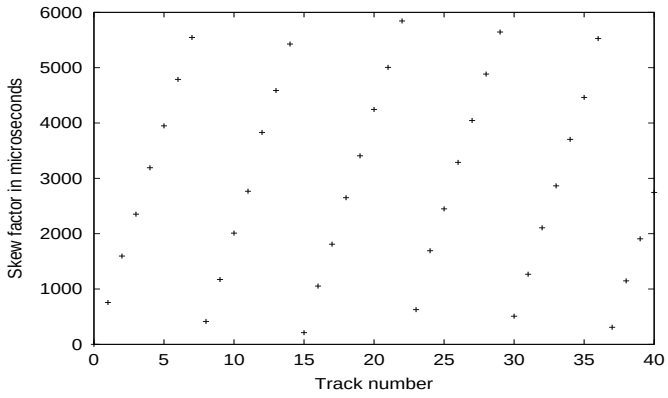


Figure 17: Skew factor s from Table 4 for ST39102LW.

From Equation 13, in order to predict the rotational delay accurately, we need to extract the skew factor (defined in Equation 10) for each track. Sample results for the extracted skew factors (in μs) are presented in Table 4, wherein we also present the LBAs for blocks residing on the same disk radius (LBA_{rot0}).

In Figure 17, we plot the skew times against track numbers. We notice a distinct trend in skew times, a property which enables us to compress this information effectively and to reduce its space requirement. In Figure 17, we also notice a slight deviation from the normal trend for tracks 12,

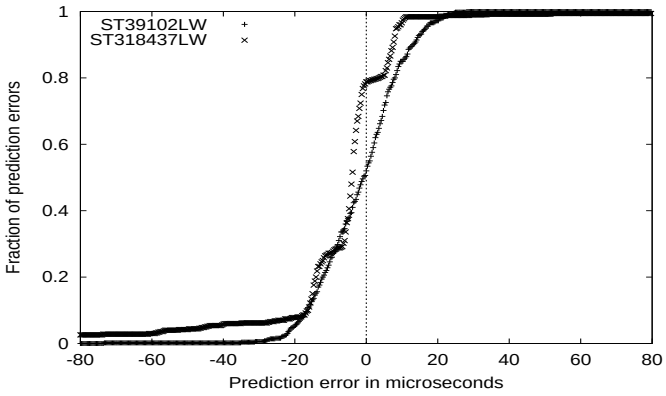


Figure 18: Rotational delay prediction accuracy for ST39102LW and ST318437LW.

24 and 36. This is due to cylinder skew, which occurs when the next track falls on an adjacent cylinder instead of the same cylinder. The experimental disk had exactly 12 surfaces. Hence, we expect a trend deviation on tracks that are multiples of 12 to account for an increased switching overhead.

Based on Equation 13 and the compressed information about skew times above, we were able to predict the rotational delay between two disk accesses. In Figure 18, we present the error distribution of rotational delay predictions for a large number of random request-pairs. We note that for the SMP-like testbed (testbed 1), which has a very predictable distribution of OS delay variations (Figure 4), our prediction is accurate within 25 μs for 99% of the requests. Even for the workstation-like testbed (testbed 2), which has less predictable OS delay variations (Figure 5), our prediction is accurate within 80 μs for 99% of the requests. These errors are negligible compared to variations in seek time, which are of the order of a millisecond. We thus conclude that with detailed disk parameters, systems can implement very accurate mechanisms for predicting rotational delays. We used seek time and rotational delay predictions from Diskbench to predict disk access times in the implementation of [3].

4.6 Read Cache

Figures 19 and 20 present results for the extraction of the cache segment size, using the method explained in Section 3.6.1. Both disks stop prefetching when they fill up the first cache segment. The extracted sizes for testbeds 1 and 2 are 561 and 204 blocks respectively. We also note that both disks continue prefetching into the next available cache segment, when they detect a long

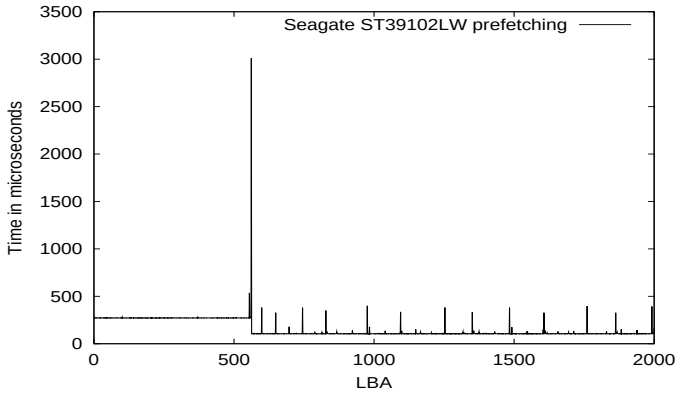


Figure 19: Read request completion times for ST39102LW.

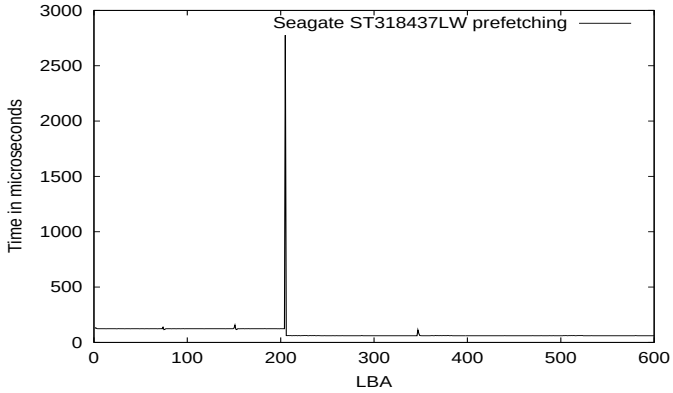


Figure 20: Read request completion times for ST318437LW.

sequential access.

Using the extracted cache segment size, we can find out the number of cache segments in the disk read cache, as explained in Section 3.6.2. Using this method, we detected three cache segments for testbed 1, and 16 segments for testbed 2.

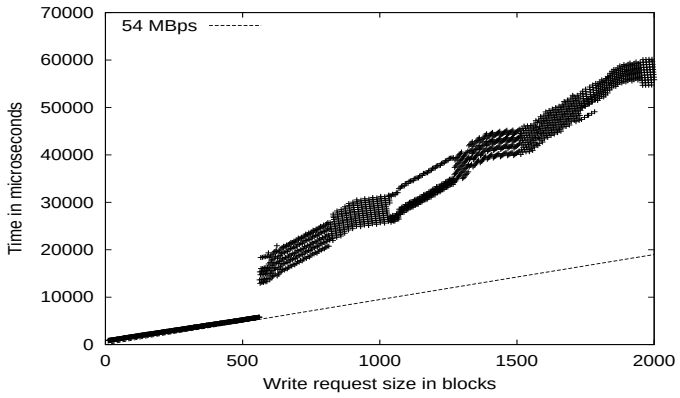


Figure 21: Write request completion times for ST39102LW.

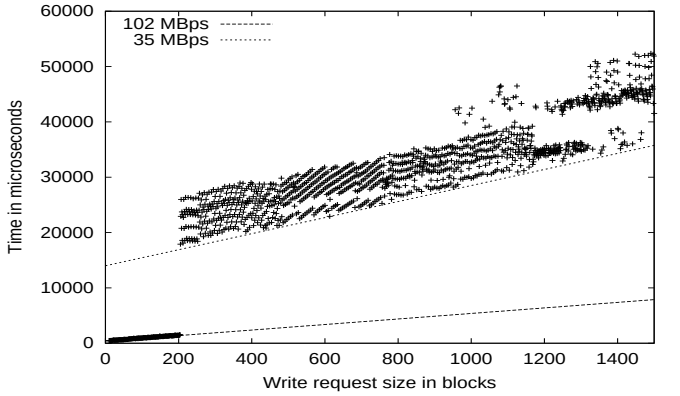


Figure 22: Write request completion times for ST318437LW

4.7 Write Buffer

Figures 21 and 22 present the results for write buffer size extraction using the method presented in Section 3.7.

The write buffer size for testbed 1 and testbed 2 were measured to be 561 and 204 blocks respectively. Comparing these to the cache segment sizes extraction presented in Section 4.6, we can see that both disks use exactly one cache segment as a write buffer.

Using these measurements we can also measure the write throughput, both to the disk write

buffer (slope of the curve for write request sizes that fit into write buffer), and to the disk platter (slope for request sizes greater than the write buffer size). In the future we plan to extract the number of cache segments that can be used as write buffers. We believe that we can use a method similar to the method we used for detecting the number of read cache segments presented in Section 3.6.2.

5 Related Work

Several previous studies have focused on the problem of disk feature extraction. The studies of [13, 17] extract disk features in order to model disk drives accurately. Both studies rely on interrogative SCSI commands to extract the LBA-to-PBA mapping from the disk. The study of [15] presents several methods for empirical feature extraction, including an approximate mapping extraction method. In [1], the authors propose methods for obtaining detailed temporal characteristics of disk drives, including methods for predicting rotational delay.

Scheduling algorithms in [7, 8, 10, 16] assume the ability to predict the rotational delay between successive requests to the disk. In [9] authors implemented their scheduler outside of disk firmware relying on their closed-source disk profiler.

Our study includes both interrogative and empirical methods for the accurate extraction of disk mapping information. The empirical methods can be used for disks that do not support interrogative commands in order to extract disk mapping information. Using empirical extraction, we can obtain accurate disk mapping information, including precise positions of track and cylinder boundaries. However, empirical methods are slower than interrogative ones. To predict rotational delay between disk IOs, Diskbench uses an approach that is similar to the approach presented in [1]. Methods in [1] are continuously keeping track of the disk head position. However, we concentrate on predicting rotational distance between two LBAs and do not require the head position knowledge, as explained in Section 3.5 and Section 4.5. Such prediction capability can be used for scheduling requests in real systems, where requests are known to arrive in a bursty fashion [12]. Diskbench can extract all the disk features that are required in order to be able to predict access time with high accuracy. We have successfully used access time prediction in [3].

Diskbench is open source and available for download [5]. At this time, we are not aware of any other disk feature extraction tool which is currently available for download as open source, and

which runs on a widely available operating system without any kernel modifications.

6 Conclusion

In this paper, we have presented Diskbench, a user-level tool for disk feature extraction. Diskbench uses both interrogative and empirical methods to extract disk features. The empirical methods extract accurate track and cylinder boundaries, track and cylinder skew times, the number of tracks per cylinder, the track sizes (in logical blocks), and the read cache and write buffer parameters. Diskbench can also extract disk features necessary for predicting the rotational delay between any two blocks on the disk. This prediction was found to be highly accurate upon validation.

We believe that this work can be used by system and application programmers to improve and guarantee disk performance. Using knowledge about disk features provided by Diskbench, system or application programmers can fine-tune disk accesses to match application requirement and can predict the disk performance, which is necessary for real-time disk scheduling. We plan to further our research in two directions. First, we plan to provide automatic disk profiling for existing multimedia real-time file systems (for example [2]). Second, we plan to implement our user-level admission control from [4] and Virtual IO [3] in Linux kernel space.

References

[1] M. Aboutabl, A. Agrawala, and J.-D. Decotignie. Temporally determinate disk access: An experimental approach. *Univ. of Maryland Technical Report CS-TR-3752*, 1997.

[2] P. Bosch and S. J. Mullender. Real-time disk scheduling in a mixed-media file system. *IEEE RTAS*, May 2000.

[3] Z. Dimitrijevic, R. Rangaswami, and E. Chang. Virtual IO: preemptible disk access (poster). *ACM Multimedia*, December 2002.

[4] Z. Dimitrijevic, R. Rangaswami, and E. Chang. The XTREAM multimedia system. *IEEE Conference on Multimedia and Expo*, August 2002.

[5] Z. Dimitrijevic, R. Rangaswami, E. Chang, D. Watson, and A. Acharya. Diskbench. *http://www.cs.ucsb.edu/∼zoran/diskbench/*, 2002.

[6] Z. Dimitrijevic, D. Watson, and A. Acharya. Scsibench. *http://www.cs.ucsb.edu/∼zoran/scsibench/*, March 2000.

[7] L. Huang and T. Chiueh. Implementation of a rotation-latency-sensitive disk scheduler. *SUNY at Stony Brook Technical Report*, May 2000.

[8] D. M. Jacobson and J. Wilkes. Disk scheduling algorithms based on rotational position. *HPL Technical Report*, February 1991.

[9] C. R. Lumb, J. Schindler, and G. R. Ganger. Freeblock scheduling outside of disk firmware. *Usenix Conference on File and Storage Technologies*, January 2002.

[10] C. R. Lumb, J. Schindler, G. R. Ganger, and D. F. Nagle. Towards higher disk head utilization: Extracting free bandwith from busy disk drives. *Proceedings of the OSDI*, 2000.

[11] M. McKusick, W. Joy, S. Leffler, and R. Fabry. A fast file system for UNIX. *ACM Transactions on Computer Systems 2*, 3:181–197, August 1984.

[12] C. Ruemmler and J. Wilkes. UNIX disk access patterns. *Usenix Conference*, pages 405–420, Winter 1993.

[13] J. Schindler and G. R. Ganger. Automated disk drive characterization. *CMU Technical Report CMU-CS-00-176*, December 1999.

[14] Seagate. Scsi interface, product manual 2. *http://www.seagate.com/support/disc/manuals/scsi/38479j.pdf*, April 1999.

[15] N. Talagala, R. H. Arpaci-Dusseau, and D. Patterson. Microbenchmark-based extraction of local and global disk characteristics. *UC Berkeley Technical Report*, 1999.

[16] B. L. Worthington, G. R. Ganger, and Y. N. Patt. Scheduling algorithms for modern disk drives. *Proceedings of the ACM Sigmetrics*, pages 241–251, May 1994.

[17] B. L. Worthington, G. R. Ganger, Y. N. Patt, and J. Wilkes. Online extraction of SCSI disk drive parameters. *Proc. of ACM Sigmetrics Conference*, pages 146–156, 1995.